

**ARSITEKTUR TUGAS BESAR
KEAMANAN SERVER & JARINGAN**

**PERANCANGAN DAN IMPLEMENTASI WEB SERVER AMAN
BERBASIS CONTAINER MENGGUNAKAN DOCKER, NGINX,
DAN MARIADB PADA APLIKASI LARAVEL**

Dosen mata kuliah: Ainul Hizriadi, S.Kom., M.Sc



Disusun Oleh:

Vincent Jose Christian Andreas Simbolon	(241402039)
Aldrik Noel Sianipar	(241402049)
Leondo Admiral Purba	(241402053)
Rafi Andara Nasution	(241402095)
Wira Hari Pratama	(241402111)
Yosia Marcel Koreshy Simanjuntak	(241402114)

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI
UNIVERSITAS SUMATERA UTARA**

2026

1. Ringkasan Eksekutif Sistem

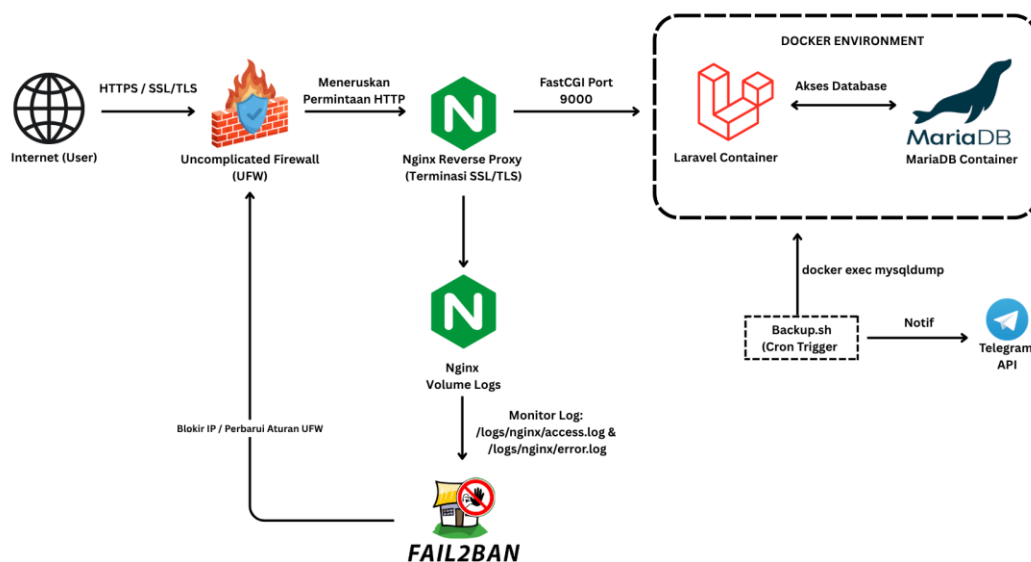
Dokumen ini menyajikan arsitektur sistem dan infrastruktur jaringan yang dirancang untuk menjalankan aplikasi berbasis Laravel secara aman, efisien, dan modular. Proyek ini diimplementasikan menggunakan pendekatan arsitektur 3-tier yang diwadahi oleh Docker Container dan diperkuat dengan konfigurasi keamanan tingkat tinggi (hardening) di level sistem operasi (Host OS) maupun jaringan.

Tiga lapis (tiers) utama sistem ini terdiri atas:

- **Tier 1 - Presentation (Nginx Proxy):** Bertindak sebagai pintu gerbang utama yang menangani terminasi SSL/TLS, reverse proxy ke server aplikasi, dan perlindungan lalu lintas HTTP/HTTPS. Menggunakan image Nginx unprivileged demi meminimalisasi hak akses eksploitasi.
- **Tier 2 - Application (Laravel App):** Wadah utama aplikasi Laravel yang menjalankan PHP-FPM 8.3 sebagai user non-root 'laravel' (UID 1000) dan mengeksekusi logika bisnis aplikasi. Berada di dalam dua jaringan terisolasi untuk menjembatani komunikasi data.
- **Tier 3 - Data (MariaDB Database):** Menyimpan data aplikasi secara persisten. Database ini sama sekali tidak mengekspos port ke luar Host OS dan hanya dapat diakses secara lokal oleh kontainer Laravel di jaringan internal.

2. Topologi Arsitektur 3-Tier

Untuk memberikan gambaran yang jelas mengenai interaksi antarkomponen, lalu lintas data, dan batas isolasi jaringan, diagram topologi sistem berikut disematkan di bawah ini:



Gambar 2.1: Diagram Topologi Jaringan & Keamanan Proyek Kelompok 06

Aliran lalu lintas data (Traffic Flow) pada sistem ini adalah sebagai berikut:

- **Aliran Web (HTTP/HTTPS):** Permintaan dikirim dari browser pengguna di internet ke alamat IP server pada port HTTPS (443). UFW Firewall memvalidasi paket, dan aturan kompatibilitas Docker-UFW meneruskannya ke kontainer Nginx Proxy (kel06-proxy). Nginx melakukan negosiasi SSL/TLS dan meneruskan permintaan secara internal ke kontainer aplikasi Laravel (kel06-app) via FastCGI di port 9000.
- **Aliran SSH (Remote Management):** Administrator melakukan koneksi remote menggunakan kunci privat (autentikasi kata sandi dinonaktifkan) menuju port kustom 2206. Firewall UFW menerapkan pembatasan laju koneksi (Rate Limit) maksimal 6 koneksi dalam 30 detik untuk mencegah serangan brute-force.
- **Aliran Database (Backend):** Kontainer aplikasi berkomunikasi dengan MariaDB di port 3306 melalui jembatan jaringan backend-net. Port 3306 MariaDB sama sekali tidak diekspos ke Host OS atau internet luar, sehingga mencegah penyerangan langsung ke database.

3. Detail Service Docker (Container Architecture)

Sistem ini dibangun menggunakan Docker Compose dengan pemisahan network dan hak akses yang ketat. Berikut adalah rincian konfigurasi kontainer yang berjalan pada lingkungan Docker:

Nama Service	Image Base	Port (Host -> Intern)	Isolasi Network	Fitur Hardening & Volume
kel06-proxy	nginxinc/nginx-unprivileged:alpine	80 -> 8080 (HTTP) 443 -> 8443 (HTTPS)	frontend-net	- read_only: true (filesystem immutable) - no-new-privileges: true - tmpfs untuk /tmp & cache - Mount SSL cert (read-only) - Mount nginx logs ke host
kel06-app	Custom build (php:8.3-fpm-alpine)	9000 -> 9000 (tidak diekspose)	frontend-net backend-net	- User non-root: 'laravel' (UID 1000) - no-new-privileges: true - Mount kode aplikasi & .env - Folder storage & cache diberi akses 775
kel06-db	mariadb:10.11	3306 (internal saja)	backend-net	- Port tidak diekspose ke host - no-new-privileges: true - Volume database persisten: db-data - Memuat env sensitif via .env

Penerapan dual-network bridge (frontend-net dan backend-net) memastikan bahwa kontainer database MariaDB (kel06-db) terisolasi penuh dari kontainer proxy (kel06-proxy). Satu-satunya komponen yang terhubung ke kedua jaringan adalah kontainer aplikasi Laravel (kel06-app), yang bertindak sebagai mediator aman.

4. Hardening Keamanan Jaringan & OS

4.1 Kebijakan UFW Firewall & Kompatibilitas Docker

Secara default, Docker memanipulasi aturan IPTables secara langsung dan dapat melompati aturan UFW. Untuk memitigasi celah keamanan ini, sistem ditambahkan aturan khusus pada berkas `/etc/ufw/after.rules` yang menyalurkan seluruh lalu lintas Docker-User ke dalam filter input dan forward milik UFW. Dengan demikian, IP yang diblokir oleh UFW maupun Fail2Ban akan otomatis diblokir pula dari kontainer Docker.

Kebijakan dasar UFW dikonfigurasi sebagai berikut:

- **Default Incoming: DENY** - Mencegah seluruh koneksi masuk yang tidak diizinkan secara eksplisit.
- **Default Outgoing: ALLOW** - Memungkinkan sistem operasi melakukan pembaruan paket dan keluar ke internet.
- **Port Terbuka:** Membuka port 80 (untuk redirect otomatis ke HTTPS), port 443 (layanan utama HTTPS), dan port 2206 (port SSH kustom).

4.2 OS Kernel Hardening (sysctl)

Hardening sistem operasi dilakukan di level kernel melalui penulisan parameter keamanan pada berkas `/etc/sysctl.d/99-kel06-hardening.conf`. Parameter ini meminimalkan kerentanan terhadap serangan denial of service (DoS), spoofing IP, dan eksploitasi memori:

Parameter Kernel	Nilai	Deskripsi & Dampak Keamanan
net.ipv4.tcp_syncookies	1	Mencegah serangan SYN Flood dengan memvalidasi TCP handshake via cookies.
net.ipv4.conf.all.accept_source_route	0	Menolak IP source routing untuk mencegah IP spoofing oleh penyerang.

net.ipv4.conf.all.rp_filter	1	Mengaktifkan Reverse Path Filtering untuk memverifikasi keaslian IP sumber.
net.ipv4.conf.all.accept_redirects	0	Menolak ICMP redirect untuk mencegah serangan Man-in-the-Middle (MitM).
kernel.kptr_restrict	2	Menyembunyikan alamat pointer kernel dari user non-root untuk mencegah eksploitasi kernel.
kernel.randomize_va_space	2	Mengaktifkan Address Space Layout Randomization (ASLR) secara penuh untuk mencegah memory corruption.
net.ipv4.tcp_fin_timeout	15	Mengurangi waktu tunggu sockets TIME_WAIT untuk mengoptimalkan sumber daya server.
kernel.dmesg_restrict	1	Menyembunyikan log kernel dmesg dari user non-root demi membatasi kebocoran informasi internal.
kernel.sysrq	0	Menonaktifkan kontrol SysRq key dari keyboard untuk melindungi sistem dari interupsi fisik.
kernel.perf_event_paranoid	3	Membatasi akses kernel performance events bagi non-privileged user guna mitigasi side-channel attack.
net.ipv4.icmp_echo_ignore_broadcasts	1	Mengabaikan permintaan ICMP broadcast untuk memproteksi server dari serangan Smurf.

net.ipv4.icmp_ignore_bogus_error_responses	1	Melindungi server dari pencatatan log akibat spoofed/bogus error responses.
net.ipv4.conf.all.log_martians	1	Mencatat paket jaringan yang mencurigakan (spoofed, source-routed, atau redirected) ke log kernel.
net.ipv4.tcp_tw_reuse	1	Mengizinkan penggunaan kembali soket TCP TIME_WAIT untuk menghemat alokasi port.
net.ipv4.tcp_max_syn_backlog	2048	Meningkatkan batasan kapasitas antrean SYN backlog untuk bertahan dari DoS SYN Flood.
net.ipv4.tcp_synack_retries	2	Mengurangi jumlah upaya transmisi ulang paket SYN+ACK untuk drop koneksi bermasalah lebih cepat.

4.3 Hardening Akses SSH Daemon

Konfigurasi SSH daemon diatur secara ketat melalui `/etc/ssh/sshd_config` dengan rincian:

- **Port Kustom (2206):** Mengubah port default 22 ke port kustom 2206 untuk meminimalisasi pemindaian bot otomatis.
- **PasswordAuthentication no:** Memaksa login menggunakan kunci SSH publik (private-public keypair) dan menolak penggunaan sandi teks biasa.
- **PermitRootLogin no:** Menutup akses login superuser secara langsung demi mencegah serangan brute-force langsung ke target root.
- **Izin Akses Ketat:** Menjalankan penetapan izin akses (`chmod 750` pada home direktori pengguna, `chmod 700` pada folder `.ssh`, dan `chmod 600` pada `authorized_keys`).

4.4 Sistem Pencegahan Intrusi (Fail2Ban Jails)

Fail2Ban diaktifkan untuk memindai log sistem dan log kontainer secara berkala. Jika aktivitas brute-force terdeteksi, IP sumber akan diblokir menggunakan UFW selama 1 jam secara inkremental (semakin sering mencoba, durasi blokir dilipatgandakan hingga maksimal 1 minggu).

Nama Jail	Log yang Dipantau	Batas Toleransi	Dampak / Tindakan
-----------	-------------------	-----------------	-------------------

(Maxretry)			
sshd	/var/log/auth.log	3 kali gagal dalam 10 menit	IP diblokir via UFW untuk port 2206.
nginx-http-auth	/logs/nginx/error.log	5 kali gagal	IP diblokir dari akses web proxy.
nginx-limit-req	/logs/nginx/error.log	10 kali limit terlampaui	Mencegah serangan DDoS/brute-force berkecepatan tinggi.
nginx-botsearch	/logs/nginx/access.log	5 kali error 404	Blokir 24 jam untuk bot yang memindai file sensitif (.env, admin).
nginx-badbots	/logs/nginx/access.log	3 kali bad request	Blokir 24 jam untuk request HTTP yang cacat/mencurigakan.
recidive	/var/log/fail2ban.log	3 kali terkena ban biasa	Eskalasi pemblokiran selama 1 minggu (7 hari) untuk IP persisten.

4.5 Konfigurasi SSL/TLS & PFS (Perfect Forward Secrecy)

Untuk mengamankan komunikasi data antara Client dan Nginx Proxy, sertifikat SSL/TLS dikonfigurasi menggunakan standar enkripsi tinggi:

- **Kunci Privat:** Dibuat dengan kekuatan enkripsi RSA 4096-bit (menggunakan skrip generate-ssl.sh).
- **Sertifikat Self-Signed:** Menggunakan hashing SHA-256 dan menyertakan Subject Alternative Names (SAN) agar kompatibel dengan validasi browser modern.
- **Perfect Forward Secrecy:** Menggunakan Diffie-Hellman parameters sebesar 2048-bit (dhparam.pem) untuk memastikan Perfect Forward Secrecy (PFS), sehingga kunci sesi

komunikasi di masa lalu tidak dapat didekripsi meskipun kunci privat server bocor di kemudian hari.

5. Prosedur Backup & Kebijakan Data

Kebijakan data diatur agar terjamin ketersediaannya (availability) serta keutuhannya (integrity) jika terjadi kegagalan sistem. Manajemen backup harian diimplementasikan secara otomatis melalui script `backup.sh` yang terintegrasi dengan Cron daemon Host OS.

Mekanisme pelaksanaan backup dirancang dengan tahapan aman berikut:

- **Penjadwalan (Cron):** Cron Job dijalankan setiap hari pukul 01:00 WIB menggunakan perintah ``/bin/bash /root/scripts/backup.sh >> /var/log/project-backup.log 2>&1``.
- **Backup Database Konsisten:** Mengakses kontainer database MariaDB menggunakan docker exec untuk menjalankan `mysqldump` dengan argumen ``--single-transaction --routines --triggers --events --quick --add-locks``. Hal ini menjamin dump database konsisten tanpa perlu mengunci (lock) tabel yang sedang aktif digunakan pengguna.
- **Backup Konfigurasi:** Menyalin file konfigurasi vital seperti ``.env``, ``docker-compose.yml``, dan seluruh isi folder ``docker/`` ke dalam direktori backup kerja.
- **Kompresi Gzip:** Seluruh file cadangan digabungkan dan dikompresi ke dalam arsip berformat ``.tar.gz`` dengan timestamp penamaan unik.
- **Integritas Checksum:** Menghasilkan file checksum SHA256 (``.sha256``) untuk memverifikasi integritas file arsip saat proses pemulihan (restore).
- **Izin Akses Arsip:** Mengubah izin akses file arsip backup secara ketat menjadi ``chmod 600`` (hanya pemilik/root yang dapat membaca dan menulis) demi mencegah pencurian file dump database.
- **Kebijakan Retensi:** Sistem secara otomatis menyisakan file backup harian selama 7 hari ke belakang (`RETENTION_DAYS=7`) dan menghapus arsip yang lebih lama secara otomatis untuk menghemat ruang disk penyimpanan server.
- **Notifikasi Telegram & Email:** Jika backup berhasil atau gagal, skrip akan mengirimkan notifikasi instan berupa ringkasan data (nama file, checksum, ukuran file) ke bot chat Telegram atau alamat email administrator.